

Atividade: Derivação de um Código a partir da Gramática de uma Linguagem de Programação

Objetivo

Pesquisar e analisar a gramática formal de uma linguagem de programação, identificando suas principais regras de produção e utilizando-as para realizar a derivação de um trecho de código válido.

A atividade tem como objetivo relacionar os conceitos de gramáticas formais, símbolos terminais e não terminais, produções e derivação com linguagens de programação reais.

Enunciado

Escolha uma linguagem de programação de sua preferência, como Python, Java, JavaScript, C, C++, C#, entre outras.

Pesquise na Internet a gramática formal da linguagem escolhida. A gramática não precisa ser analisada integralmente: você pode selecionar apenas as regras necessárias para representar um pequeno programa ou trecho de código.

A partir da gramática encontrada, realize as seguintes etapas:

Identifique a fonte da gramática

Informe a linguagem escolhida.

Apresente o endereço da documentação ou da fonte consultada.

Identifique qual notação é utilizada, por exemplo, BNF, EBNF, PEG ou outra.

Selecione as produções

Escolha um pequeno conjunto de regras da gramática que permita representar o código que será derivado.

Explique brevemente o significado das principais regras utilizadas.

Defina o código a ser gerado

Escolha um pequeno trecho de código válido na linguagem.

O código deve ser suficientemente simples para que seja possível demonstrar sua derivação.

Recomenda-se utilizar uma construção como uma atribuição, uma expressão aritmética, uma estrutura condicional ou uma chamada de função.

Realize a derivação

Comece pelo símbolo inicial da gramática.

Aplique sucessivamente as produções escolhidas.

Mostre cada etapa da derivação até chegar ao código ou à sequência de tokens correspondente ao código escolhido.

Apresente o resultado

Mostre o código final gerado.

Explique, com suas palavras, como as regras da gramática foram utilizadas para chegar ao código.

Indique quais símbolos são terminais e quais são não terminais nas regras utilizadas.

Exemplo simplificado

Considere uma gramática hipotética:

```
<expressao> ::= <numero> | <numero> "+" <numero>  
<numero> ::= "1" | "2" | "3"
```

Deseja-se derivar:

1 + 2

A derivação pode ser apresentada como:

```
<expressao>  
⇒ <numero> "+" <numero>  
⇒ "1" "+" <numero>  
⇒ "1" "+" "2"
```

Portanto:

1 + 2

O objetivo da atividade é realizar esse mesmo processo utilizando uma gramática real de uma linguagem de programação.

## Tópico 1: Identificação da Fonte da Gramática:

**Site:** <https://docs.oracle.com/javase/specs/jls/se17/html/jls-19.html>

**Notação utilizada:** Variação de EBNF (Extended Backus-Naur Form) adotada pela Oracle na especificação do Java.

## Tópico 2: Código gerado

```
public static double Somar(double valorDeFuncao){  
  
    valorDeFuncao = 110;  
  
    return ((valorDeFuncao + 10) / 2);  
}
```

## Tópico 3: Regras gramaticais utilizadas

<method\_declaration> ::= <modifiers> <type> <identifier> ( <parameter> ) <method\_body>  
<modifiers> ::= public static  
<type> ::= double  
<parameter> ::= <type> <identifier>  
<method\_body> ::= { <statement\_list> }  
<statement\_list> ::= <statement> <statement>  
<statement> ::= <assignment\_statement> | <return\_statement> <assignment\_statement> ::=  
<identifier> = <expression> ; <return\_statement> ::= return <expression> ;  
<expression> ::= ( ( <identifier> + <integer\_literal> ) / <integer\_literal> ) | <integer\_literal>  
<integer\_literal> ::= 110 | 10 | 2  
<identifier> ::= Somar | valorDeFuncao

## Tópico 4: Derivação

<method\_declaration>  
-> <modifiers> <type> <identifier> (<parameter>) <method\_body>  
-> public static <type> <identifier> (<parameter>) <method\_body>  
-> public static double <identifier> (<parameter>) <method\_body>  
-> public static double Somar (<parameter>) <method\_body>  
-> public static double Somar (<type> <identifier>) <method\_body>  
-> public static double Somar (double valorDeFuncao) <method\_body>  
-> public static double Somar (double valorDeFuncao) {<statement\_list>}  
-> public static double Somar (double valorDeFuncao) { <statement> <statement>}  
-> public static double Somar (double valorDeFuncao) {<assignment\_statement> <statement>  
>}  
-> public static double Somar (double valorDeFuncao) { <identifier> = <expression>;  
<return\_statement> }  
-> public static double Somar (double valorDeFuncao) { valorFuncao = <expression>;  
<return\_statement>}  
-> public static double Somar (double valorDeFuncao) { valorFuncao = <integer\_literal>;  
<return\_statement>}

```
-> public static double Somar (double valorDeFuncao) { valorFuncao = 110;
<return_statement>}
-> public static double Somar (double valorDeFuncao) { valorFuncao = 110; return
<expression>}
-> public static double Somar (double valorDeFuncao) { valorFuncao = 110; return
((<identifier> + 10)/2)}
->public static double Somar (double valorDeFuncao) { valorFuncao = 110; return
((valorDeFuncao + 10)/2)}
```

## **Tópico 5: Breve explicação textual**

Em Java, o método estático apresentado é estruturado a partir de seus modificadores (public static), que indicam que ele é acessível publicamente e pertence à classe, dispensando a instanciamento de um objeto para ser executado. O tipo de retorno double define que a função sempre devolverá um número de ponto flutuante de dupla precisão, enquanto o identificador Somar nomeia a rotina que recebe um parâmetro de entrada denominado valorDeFuncao, também do tipo double.

No corpo do método, delimitado por chaves, ocorrem duas instruções sequenciais: primeiro, a variável valorDeFuncao tem seu conteúdo sobrescrito pelo valor literal 110, o que faz com que qualquer argumento original passado na chamada do método seja descartado e substituído. Em seguida, a instrução de retorno avalia a expressão aritmética ((valorDeFuncao + 10) / 2). Respeitando a precedência dos parênteses, o programa primeiro soma 110 com 10 (totalizando 120) e depois divide esse total por 2, fazendo com que o método devolva invariável e deterministamente o valor 60.0 ao final de sua execução.